

AR-Quivers of Finite-Dimensional Algebras via GAP

Quick Start

Launch the container

Important

A VPN is required to access the binder when you are in China.

Click the badge to launch automatically: , or open one of the following links:

- Stable version: https://mybinder.org/v2/gh/zhangchenchengSJTU/GAP_AR_Quiver/main
- Testing version: https://mybinder.org/v2/gh/zhangchenchengSJTU/GAP_AR_Quiver/test

Manual launch: open [Binder](#) and select the `GitHub` option.

1. In the `GitHub repository name or URL` field, paste `https://github.com/zhangchenchengSJTU/GAP_AR_Quiver`.
2. Click `launch`. To use the testing version, set `Git ref (branch, tag, or commit)` to `test`.
3. Wait for the environment to build. Binder will redirect to the Jupyter Notebook page.

Enter Jupyter Notebook

After the environment loads, the browser address will resemble

`https://hub.bids.mybinder.org/user/zhangchenchengsjtu-gap_ar_quiver-???????/tree`. The root directory contains:

- `ARquiver`: working directory for AR-quiver computation. It contains:
 - `source`: source code directory.
 - `run.ipynb`: notebook for running computation and rendering.
 - `example.txt`, `yourfile.txt`, and similar files: input files containing quiver data.
- `Dockerfile`: environment specification for developers.
- `readme.md`, `readme.html`: documentation.

Files

Running

Clusters

Select items to perform actions on them.

0

/

Name

Last Modified

File size

Folder

ARquiver

5 分鐘前

File

Dockerfile

8 分鐘前

244 B

File

readme.html

8 分鐘前

458 kB

File

readme.md

8 分鐘前

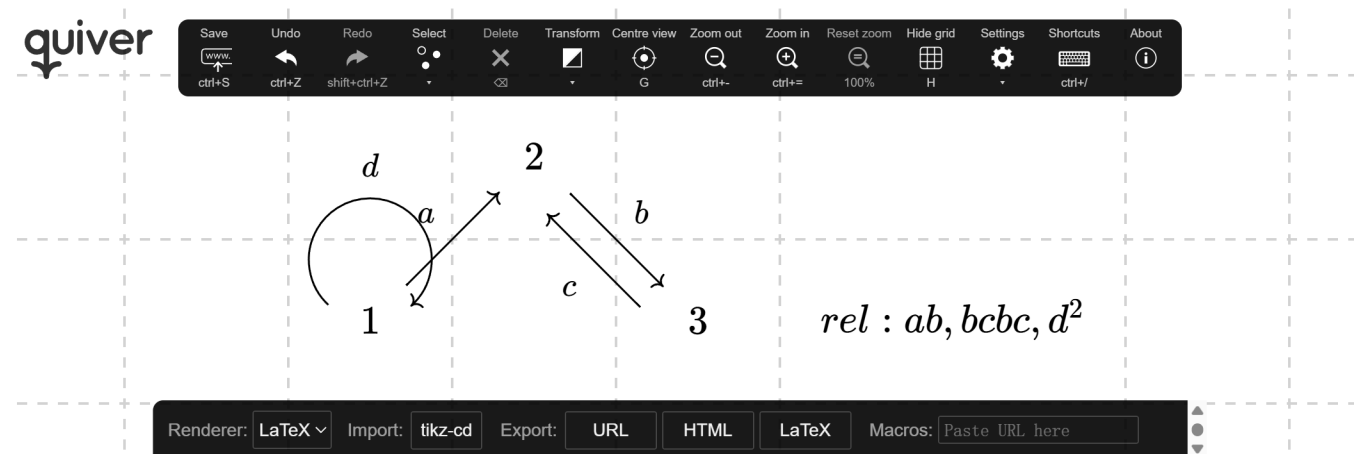
21.7 kB

Draw the path algebra

Use <https://q.uiver.app> to draw a quiver with relations, following these conventions:

- vertices are positive integers;
- arrows are single Latin or Greek letters in $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ format, e.g. `a` or `\alpha`;
- relations are entered in an empty grid cell in the form `rel: ...`.

Example:



Click `LaTeX` at the bottom of the `q.uiver` page and copy the generated `tikzcd` source code. Create a new file `yourfile.txt` in the `ARquiver` directory, paste the code into it, and save.

Draw the AR-quiver

Open `run.ipynb` and run the following cells in order:

```
# yourfile.txt -> yourfile.log
%run source/compute_all.py
```

```
# yourfile.log -> yourfile.html
%run source/render_all.py
```

The `ARquiver` directory will then contain:

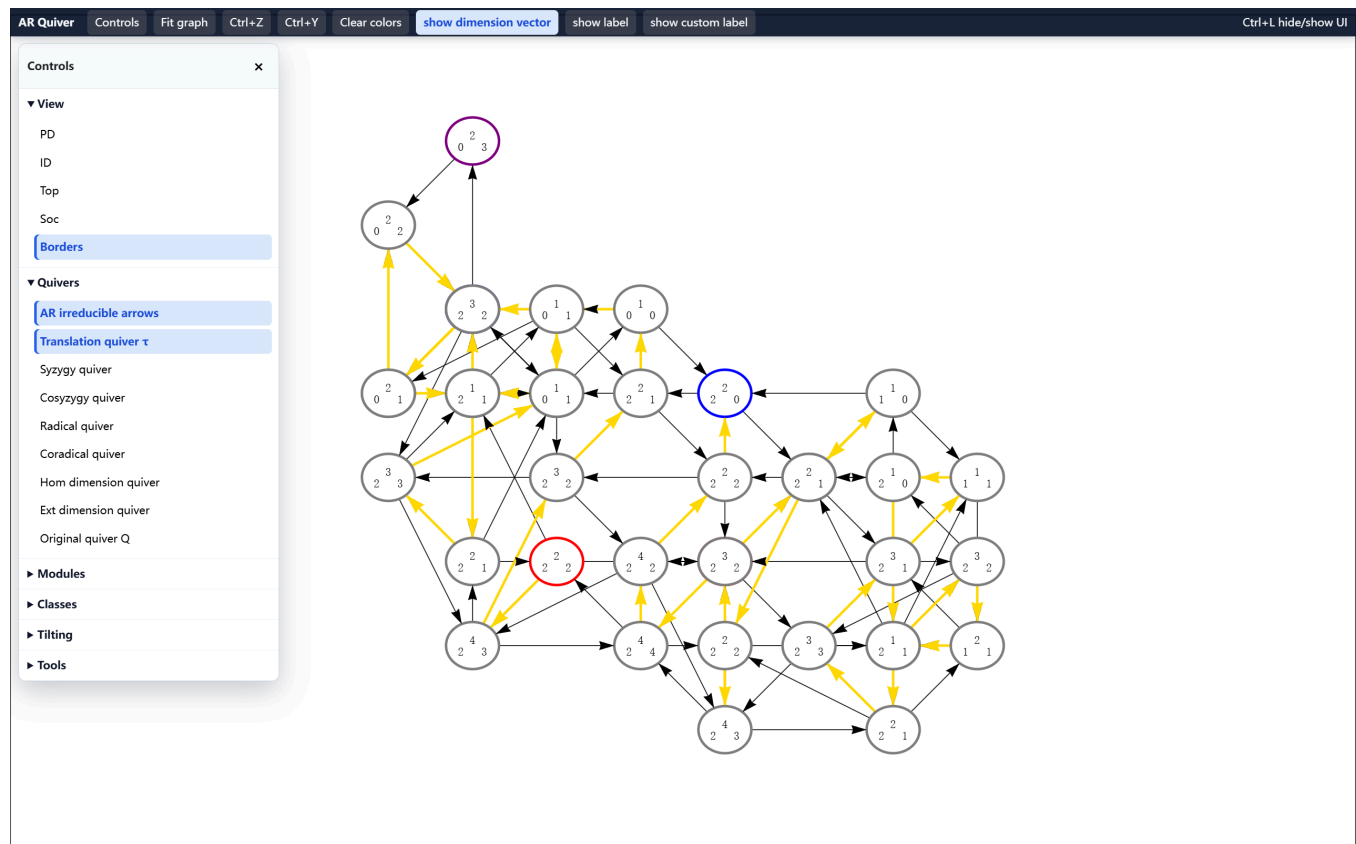
- `yourfile.log`: algebra computation log;
- `yourfile.html`: interactive AR-quiver canvas.

The `html` file can be downloaded directly:

Files			Running		Clusters	
Duplicate Move Download View Edit					Upload New	
2 / ARquiver			Name	Last Modified	File size	
..				幾秒前		
source				7 分鐘前		
run.ipynb				Running 4 分鐘前	10.9 kB	
example.html				5 分鐘前	282 kB	
example.log				5 分鐘前	39.8 kB	
example.txt				7 分鐘前	458 B	
yourfile.html				5 分鐘前	349 kB	
yourfile.log				5 分鐘前	106 kB	
yourfile.txt				7 分鐘前	476 B	

Arrange the AR-quiver into a usual form

Open `yourfile.html` to view the interactive AR-quiver.



Basic controls:

- Each ellipse represents an indecomposable module; its dimension vector is arranged according to vertex positions in the `tikzcd` input. Some ellipses may overlap.
- Purple border: projective-injective. Red border: projective non-injective. Blue border: injective non-projective.
- `Quivers > AR irreducible arrows`: show/hide irreducible morphisms (black arrows).
- `Quivers > Translation quiver  $\tau$` : show/hide the AR translation $\tau = D\text{Tr}$ (golden arrows).
- `view > Borders`: show/hide vertex borders.
- `Ctrl+Z` / `Ctrl+Y`: undo / redo.

The main task is to arrange the AR-quiver into a readable standard form.

Tip

The τ -orbits are disjoint. Projective-injective objects belong to no τ -orbit; every other indecomposable belongs to exactly one. Each τ -orbit is of one of the following types:

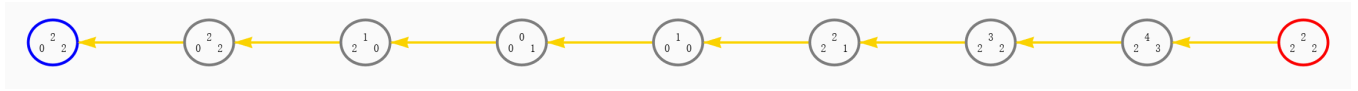
1. a path from an injective to a projective object;
2. a cycle containing no projective or injective object.

Start by identifying a projective object with a long τ -orbit, e.g., $\begin{pmatrix} 2 \\ 0 \\ 2 \end{pmatrix}$. If vertices overlap, refresh the page to obtain a cleaner initial layout. Hide `AR irreducible arrows` and place the τ -arrow $\begin{pmatrix} 2 \\ 0 \\ 2 \end{pmatrix} \xleftarrow{\tau} \begin{pmatrix} 2 \\ 0 \\ 2 \end{pmatrix}$ horizontally in an empty region.

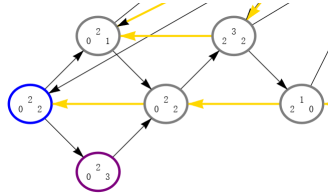
Tip

Long-press a golden arrow to align the subsequent arrow (ensure no black arrows are hidden behind it).

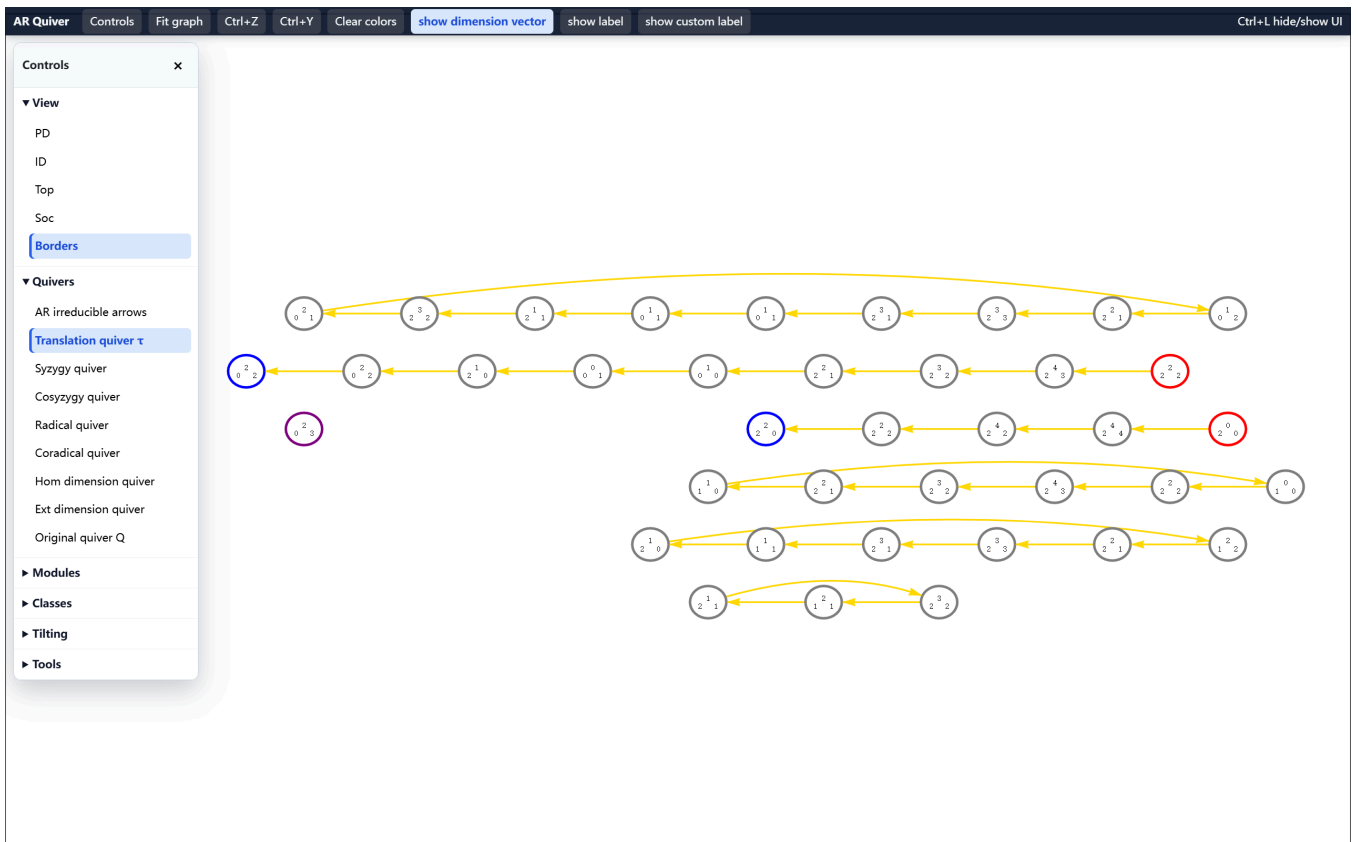
This gives:



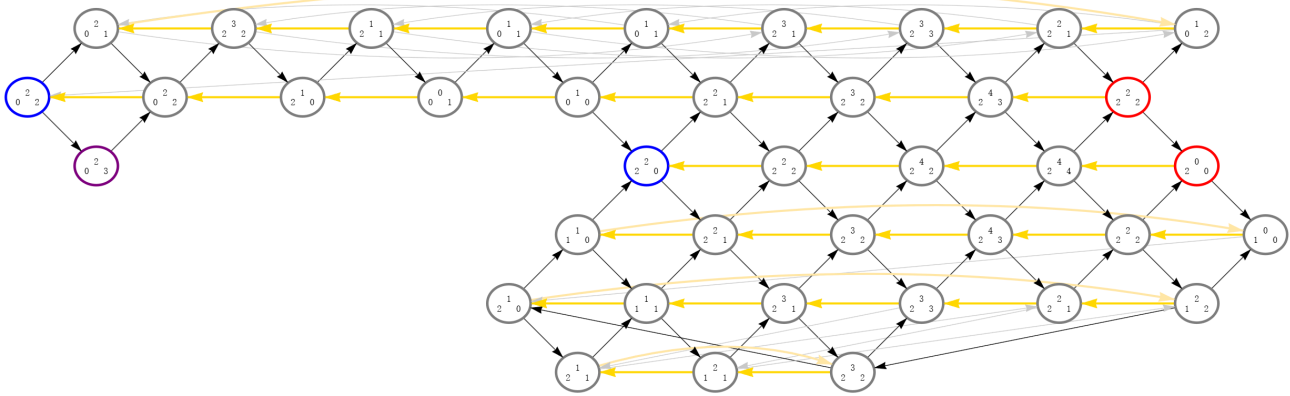
Enable **AR irreducible arrows** and locate the almost split sequence $0 \rightarrow 2 \rightarrow \bigoplus M_i \rightarrow 0 \rightarrow 2$.



Hide **AR irreducible arrows** again and long-press the golden arrows to complete the alignment. For cycles, select an edge and use $\uparrow / \downarrow$ to adjust its curvature. After these operations, the τ -orbits are obtained:



Finally, hide **AR irreducible arrows** and adjust the curvature of any black arrows previously hidden behind golden ones. Double-click an edge to toggle between dark and light colours. The final AR-quiver is:



Tip

Press **Ctrl + L** for full screen and enjoy this quiver.

Features

The generated `filename.html` is an interactive workspace for studying the AR-quiver and homological data of the algebra defined by `filename.txt`. The page consists of a main canvas, a top navigation bar, and a sidebar toggled by `controls`. The sections below follow the interface order: canvas and navigation bar first, then each sidebar section from top to bottom.

Basic canvas interaction

Each ellipse represents an indecomposable module. The label is the dimension vector, arranged according to the input quiver shape when available. The internal node number is the identifier used throughout `filename.log`.

- Drag a vertex to reposition it.
- Drag the background to pan.
- Scroll to zoom.
- Hover over a vertex to display its node number.
- Double-click an edge to toggle dark/light colour.
- Select an edge and press **↑** / **↓** to adjust its curvature.

These operations affect only the visualisation and do not recompute the algebra.

Top navigation bar

controls

Opens or hides the sidebar. The sidebar contains all overlays, lists, and tools, divided into `view`, `Quivers`, `Modules`, `Classes`, `Tilting`, and `Tools`.

Fit graph

Recentres and rescales the canvas to fit the visible graph in the window.

Ctrl+Z and Ctrl+Y

Duplicate the keyboard shortcuts: `Ctrl+Z` undoes recent layout or colouring operations; `Ctrl+Y` redoes them. Intended for manual arrangement, especially when aligning τ -orbits.

Clear colors

Removes all temporary colourings produced by the calculator, list selections, torsion/cotorsion classes, tilting objects, support τ -tilting objects, and module-class highlights. Arrows and underlying data are unaffected.

show dimension vector

Default label mode. Each vertex displays the dimension vector of the corresponding indecomposable module. The source is the `digraph Quiver` block in `filename.log`.

show label

Replaces each dimension vector with the internal node number. Useful when comparing the canvas with `filename.log`, where all tables reference modules by node number.

show custom label

Displays user-defined labels when available; falls back to dimension-vector labels otherwise.

Tip

Double-click a vertex to **customise its label**. The input supports basic Unicode. For instance, `N^2` renders as N^2 .

Ctrl+L hide/show UI

Hides or restores the user interface. Useful for screenshots or for arranging a large graph.

Sidebar: View

PD

Displays the projective dimension of each indecomposable module as a floating label; `-1` denotes ∞ or an unresolved value. Data source:

```
PDID := [[node, pd, id], ...];
```

ID

Displays the injective dimension of each module, using the third entry of the same `PDID` table.

Top

Displays the simple factors of $\text{top}(M)$ for each indecomposable module M , with multiplicities. Data source:

```
TopSoc := [[node, top, soc], ...];
```

Soc

Displays the simple factors of $\text{soc}(M)$ for each indecomposable module M , using the same **TopSoc** table.

Borders

Shows or hides vertex borders. Convention:

- blue: injective;
- red: projective;
- purple: projective-injective;
- grey: other indecomposable.

Data source:

```
Projective modules found (Node IDs): [...]
Injective modules found (Node IDs): [...]
```

Sidebar: Quivers

Overlays auxiliary quivers on the AR-quiver canvas.

AR irreducible arrows

Shows or hides the irreducible morphisms of the AR-quiver (black arrows). Data source:

```
digraph Quiver { ... }
```

Repeated arrows may be collapsed visually; multiplicity information is preserved.

Translation quiver τ

Shows the AR translation quiver. A golden arrow $M \rightarrow N$ indicates $N = \tau M$. Data source:

```
digraph TranslationQuiver { ... }
```

Syzygy quiver

Shows the syzygy quiver. A pink arrow $X \rightarrow Y$ indicates $Y \in \text{add}(\Omega X)$. Data source:

```
digraph SyzygySummand { ... }
```

The calculator iterates this quiver to compute $\Omega^n(X)$.

Cosyzygy quiver

Shows the cosyzygy quiver. A green arrow $X \rightarrow Y$ indicates $Y \in \text{add}(\Sigma X)$. Data source:

```
digraph CosyzygySummand { ... }
```

The calculator iterates this quiver to compute $\Sigma^n(X)$.

Radical quiver

Shows the radical quiver. A cyan arrow $X \longrightarrow Y$ indicates $Y \in \text{add}(\text{Rad}(X))$; repeated arrows encode multiplicities. Data source:

```
digraph RadicalSummand { ... }
```

The calculator iterates this quiver to compute $\text{Rad}^n(X)$.

Coradical quiver

Shows the coradical quiver. A purple arrow $X \longrightarrow Y$ indicates $Y \in \text{add}(X/\text{Soc}(X))$; repeated arrows encode multiplicities. Data source:

```
digraph CoradicalSummand { ... }
```

The calculator iterates this quiver to compute $\text{Corad}^n(X)$.

Hom dimension quiver

Shows nonzero Hom dimensions. A brown arrow $M \longrightarrow N$ with label k (unlabelled means $k=1$) indicates $\dim \text{Hom}(M, N) = k$. Data source:

```
digraph HomDim { ... }
```

Ext dimension quiver

Shows nonzero Ext^1 dimensions. A red arrow $M \longrightarrow N$ with label k (unlabelled means $k=1$) indicates $\dim \text{Ext}^1(M, N) = k$. Data source:

```
digraph ExtDim { ... }
```

Combined with the syzygy quiver, this data is used by the calculator to evaluate $\text{Ext}^{\geq 1}$ dimensions.

Extension middle-term table

The computation log also contains an extension middle-term table:

```
# --- ExtensionMiddleTermTable --- #
ExtensionMiddleTermTable := [
  [rec(sub := i, quot := j, mids := [...], ...), ...],
  ...
];;
```

The entry with `sub := i` and `quot := j` records known middle terms of short exact sequences

$$0 \rightarrow M_i \rightarrow E \rightarrow M_j \rightarrow 0.$$

Each row of `mids` is the list of indecomposable summands of one possible middle term. For example, `mids := [[2, 3], [1, 12]]` means that the known middle terms include both $M_2 \oplus M_3$ and $M_1 \oplus M_{12}$.

By default this table is a fast **known-middle-term** table, not a full enumeration of all extension classes. It includes:

- split extensions;
- the fact that pairs with $\dim \operatorname{Ext}^1(M_j, M_i) = 0$ have only the split middle term;
- non-split sequences coming from radical/top, socle/coradical, projective covers/syzygies, and injective envelopes/cosyzygies;
- middle terms inferred from AR translation meshes and dimension-compatible commutative squares in the AR-quiver;
- additional middle terms obtained by iteratively composing such pushout/pullback squares, using the fact that a composite of pushout/pullback squares is again a pushout/pullback square. This iteration is guarded by `max_extension_square_composition_iterations` and `max_extension_square_composition_rectangles` to avoid blow-up on large examples.

To request the slower full pushout-based table, set the GAP variable

```
compute_full_extension_table := true;;
```

before running the computation. Over an infinite field, the full mode still uses the split extension plus basis representatives produced by `ExtOverAlgebra`; over a finite field it can enumerate finite linear combinations.

original quiver Q

Opens a draggable window displaying the original quiver and its relations. Data source:

```
digraph Q { ... }
rel := ...;
```

The `see filename.txt` button opens the original input. The `open in q.uiver` button opens the URL stored in the first line of the input file.

Sidebar: Modules

These buttons highlight special classes of indecomposable modules without altering the graph.

Note

Each property below is closed under direct summands, so it suffices to state it for indecomposable modules.

Torsionless

Highlights **non-projective** torsionless modules (cyan). A module M is torsionless if the canonical map

$$M \rightarrow D^2 M, \quad m \mapsto (f \mapsto f(m))$$

is injective, or equivalently if M embeds into a projective module. Data source:

```
Torsionless modules found (Node IDs): [...]
```

Reflexive

Highlights **non-projective** reflexive modules (purple). A module M is reflexive if the canonical map

$$M \rightarrow D^2 M, \quad m \mapsto (f \mapsto f(m))$$

is an isomorphism. Data source:

```
Reflexive modules found (Node IDs): [...]
```

Gorenstein projective

Highlights **non-projective** Gorenstein projective modules (green). Data source:

```
Gorenstein projective modules found (Node IDs): [...]
```

Gorenstein injective

Highlights **non-injective** Gorenstein injective modules (red). Data source:

```
Gorenstein injective modules found (Node IDs): [...]
```

Sidebar: Classes

Torsion classes

Opens a list of torsion pairs. Each row has the form

```
T := [...] | F := [...] | tilting/non-tilting
```

Colour convention: red vertices denote the torsion class $\mathcal{T}$; green vertices denote the torsion-free class $\mathcal{F}$. For a tilting module L , the induced torsion pair satisfies

$$\mathcal{T} = \text{gen}(L) = \text{Ker Ext}^1(L, -), \quad \mathcal{F} = \text{Ker Hom}(L, -).$$

Data source:

```
# --- TorsionPairTable --- #
T := [...] | F := [...]
```

The `tilting` flag records whether the torsion pair arises from a tilting module.

⚠ Caution

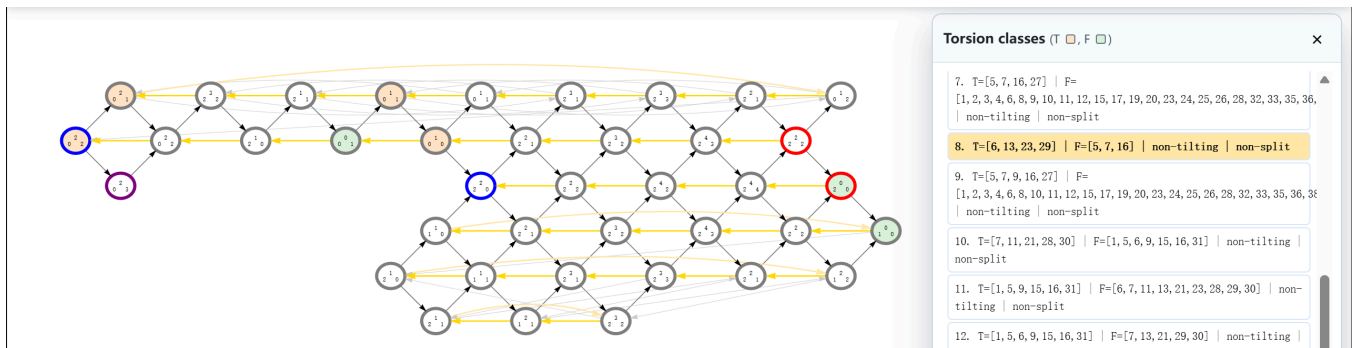
Distinct tilting modules may induce the same torsion pair.

The `split` flag records whether the torsion pair $(\mathcal{T}, \mathcal{F})$ splits, i.e., $\mathcal{T} \cup \mathcal{F}$ exhausts all indecomposable modules.

Controls > **GAP codes** > **Extension closure** provides copyable GAP code for computing extension closures from a middle-term table. This is kept out of the generated HTML sidebar list because enumerating all extension-closed classes can be exponentially large. The GAP snippet includes:

- `ExtensionClosureFromTable(X, table);`
- `IsExtensionClosedClassFromTable(X, table);`
- `AllExtensionClosedClassesFromTable(table)` for small enough examples.

The HTML still stores `ExtensionMiddleTermTable`, so the calculator can compute the closure of a selected class on demand without rendering the full list.



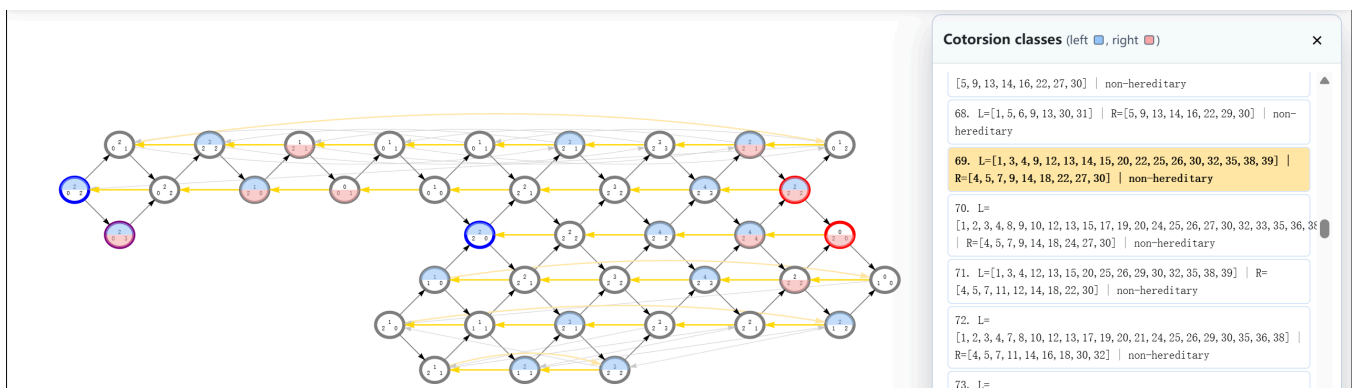
Cotorsion classes

Opens a list of cotorsion pairs. Each row has the form

$L := [...] \mid R := [...] \mid \text{Hereditary} := \text{true/false}$

Colour convention: blue vertices denote the left class $\mathcal{L}$; red vertices denote the right class $\mathcal{R}$; modules in both classes are half-coloured. The `hereditary` flag records whether $\text{Ext}^{\geq 1}(\mathcal{L}, \mathcal{R}) = 0$. Data source:

```
# --- CotorsionPairTable --- #
L := [...] | R := [...] | Hereditary := ...
```



Sidebar: Tilting

Tilting modules

Opens the list of classical tilting modules. A module T is (classical 1-) tilting if $\text{pd } T \leq 1$, $\text{Ext}^{\geq 1}(T, T) = 0$, and there exists a short exact sequence $A \twoheadrightarrow T^0 \twoheadrightarrow T^1$ with $T^0, T^1 \in \text{add}(T)$.

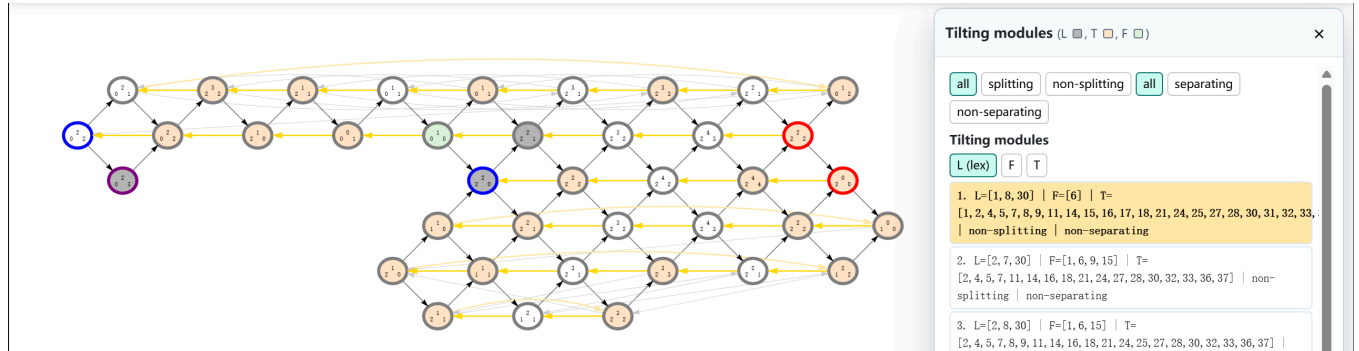
Colour convention: grey vertices are the indecomposable summands of the selected tilting module; red vertices are in the induced torsion class; green vertices are in the induced torsion-free class. Data source:

```
L := [...]
F := [...]
T := [...]
```

where **L** is the tilting module, **T** is the torsion class, and **F** is the torsion-free class.

The **splitting** tag records whether every indecomposable module in **F** has injective dimension at most one.

The **separating** tag records whether the induced torsion pair is split.



Support τ -tilting

Opens the list of support τ -tilting pairs. A pair (P, M) is support τ -tilting if P is projective, M is τ -rigid, $\text{Hom}(P, M) = 0$, and $|\text{ind } P| + |\text{ind } M|$ equals the number of vertices of the original quiver. Data source:

```
# --- SupportTauTiltingTable --- #
P := [...] | M := [...]
```

Almost support τ -tilting

Opens the list of almost support τ -tilting pairs, which have the same format as support τ -tilting pairs but with total summand count one less than the number of vertices. Data source:

```
# --- AlmostSupportTauTiltingTable --- #
P := [...] | M := [...]
```

Sidebar: Tools

calculator

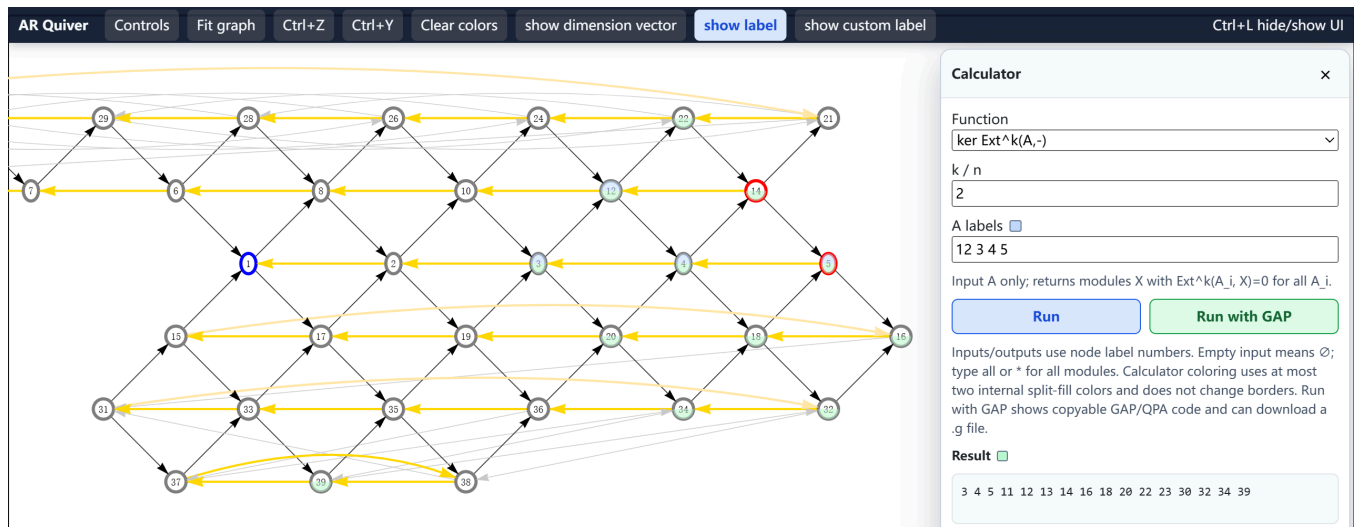
Performs operations on data stored in the HTML file. Inputs and outputs use node numbers.

Available operations:

- **dim Ext^k(A, B)**: computes $\sum_{i,j} \dim \text{Ext}^k(A_i, B_j)$ over all selected labels.
- **extension closure(A)**: computes the smallest class containing the selected labels and closed under the known middle terms in **ExtensionMiddleTermTable**.
- **middle terms A $\rightarrow$? $\rightarrow$ B**: requires one indecomposable label in **A** and one in **B**, and lists the known middle terms of short exact sequences $0 \rightarrow A \rightarrow E \rightarrow B \rightarrow 0$, one middle term per line.

- `ker Ext^k(A, -)` : returns all X with $\text{Ext}^k(A_i, X) = 0$ for every selected A_i .
- `ker Ext^k(-, B)` : returns all X with $\text{Ext}^k(X, B_j) = 0$ for every selected B_j .
- `$\Omega^n(A)$` : iterates the syzygy quiver n times.
- `$\Sigma^n(A)$` : iterates the cosyzygy quiver n times.
- `$\text{Rad}^n(A)$` : iterates the radical quiver n times.
- `$\text{corad}^n(A)$` : iterates the coradical quiver n times.

Notation: `3 + 5` denotes $M_3 \oplus M_5$; `3^2 + 5` denotes $M_3^{\oplus 2} \oplus M_5$. Empty output is displayed as `∅`.



Run with GAP

Generates copyable `GAP / QPA` code that reconstructs the quiver, algebra, projective modules, injective modules, simple modules, and serialised indecomposable modules. The script can be downloaded as a `.g` file.

Export AR-quiver to TeX

Exports the current AR-quiver layout as copyable $T_E X$ code. Two modes are available:

- `xymatrix` : matrix-style output for `xy-pic`.
- `tikz` : `tikzpicture` output preserving vertex positions, dashed τ -arrows, and curved arrows via `bend left` / `bend right`.

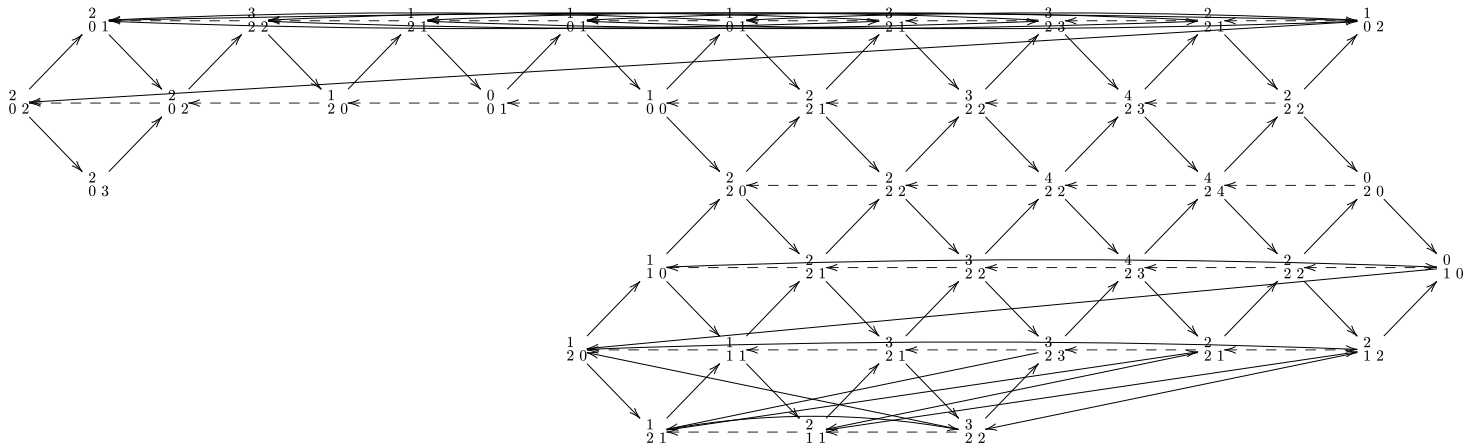
Required preamble packages:

```
\usepackage[all]{xy}           % for xymatrix
\usepackage{tikz,amsmath} % for tikz
```

⚠ Caution

Use `tikz` when the picture contains **curved dashed arrows**, as `xymatrix` has limited support for that combination.

`Typora` supports `xymatrix`.



Display code

Records the current node positions and arrow-curve offsets as copyable code. The layout can be restored later or transferred to another `html` file with the same underlying graph. Mathematical data are not included.

For instance, the `Display code` of the above diagram is

```
ARQ2.YWawCweWVVTVXVRVZVPVbVNVdVVXfXXXdXZxbXeUCUOUauQUYUSUWUUUOWUYeYWYcYYYaYVZZXZ.Oav0cv0ev0
gv0iv0kv0mv0ov0qv1KX1SX1ZX1fY
```

Color legend

Opens a draggable legend summarising all visual conventions: vertex borders, module-class colours, AR arrows, τ -arrows, syzygy, cosyzygy, radical, coradical, Hom and Ext^1 arrows, torsion/cotorsion colours, tilting colours, support τ -tilting colours, calculator colours, and floating labels.

Acknowledgements

The author thanks the developers and maintainers of [GAP](#), [Binder](#), and [quiver](#). The implementation drew in part on A. Konovalov's [try-gap-in-jupyter](#) repository. The AI model `claude-sonnet-4-6-thinking` assisted in the development of this codebase. `readme.html` was generated by [Typora](#).